Full Stack Development with MERN

Project Documentation

1. Introduction

Project Title: House Rent Management System

Team Members

Name	Role
CHINTAKULA MUKESH NARAYANA	Full Stack Developer
DASARI JEEVAN SAI CHAKRI	Frontend & Testing
DASARI TEJEESWARI	Documentation & Testing
PILLI VIJAYAKUMARI	Backend & Testing

---

2. Project Overview

Purpose

The **House Rent Management System** is a MERN Stack web application developed to simplify the process of renting houses. It provides a centralized platform where property owners can list rental properties, and users can search, view, and book houses online. The system eliminates the traditional manual rental process by providing a secure and user-friendly interface.

Features

- User Registration
- User Login using JWT Authentication
- Secure Password Encryption using bcrypt
- Add New Property
- View All Properties
- Property Details Page
- Search Properties by Location
- Filter by Property Type
- Filter by Maximum Price
- Book Rental Property
- View My Bookings

- Dashboard
- Responsive Design

---

3. Architecture

Frontend

The frontend is developed using **React.js**, which provides a component-based architecture. React Router DOM is used for navigation, Axios is used for API communication, and Bootstrap 5 is used for responsive UI design.

Frontend Components

- Navbar
- Home Page
- Login Page
- Register Page
- Property List
- Property Details
- Dashboard
- Booking Page
- Footer

Backend

The backend is developed using **Node.js** and **Express.js**.

Responsibilities include:

- User Authentication
- JWT Token Generation
- Property Management
- Booking Management
- Database Communication
- REST API Development

Main Backend Modules

- Authentication Routes
- Property Routes

- Booking Routes
- Middleware
- Controllers
- Models

Database

The application uses **MongoDB Atlas** as the cloud database.

Collections:

Users

- _id
- name
- email
- password

Properties

- title
- description
- location
- price
- propertyType
- image
- owner

Bookings

- userId
- propertyId
- bookingDate

---

4. Setup Instructions

Prerequisites

Install the following software:

- Node.js
- MongoDB Atlas Account
- Visual Studio Code
- Git
- npm

Installation

Clone Repository

```
git clone https://github.com/yourusername/House-Rent-Management-System.git
```

Backend

```
cd server
npm install
```

Create `.env`

```
PORT=5000

MONGO_URI=YourMongoDBConnectionString

JWT_SECRET=YourSecretKey
```

Start Backend

```
npm start
```

Frontend

```
cd client

npm install

npm start
```

Application runs on

```
http://localhost:3000
```

Backend runs on

```
http://localhost:5000
```

---

5. Folder Structure

```
House-Rent-Management-System
|
├── client
|   ├── public
|   ├── src
|   |   ├── components
|   |   ├── pages
|   |   ├── services
|   |   ├── App.js
|   |   └── index.js
|
├── server
|   ├── config
|   ├── controllers
|   ├── middleware
|   ├── models
|   ├── routes
|   ├── server.js
|   └── package.json
|
├── screenshots
├── README.md
└── package.json
```

---

6. Running the Application

Start Backend

```
cd server

npm install

npm start
```

Start Frontend

```
cd client

npm install

npm start
```

---

7. API Documentation

Authentication APIs

Register

```
POST /api/auth/register
```

Request

```
{
"name":"Mukesh",
"email":"mukesh@gmail.com",
"password":"123456"
}
```

Response

```
{
"message":"User Registered Successfully"
}
```

Login

```
POST /api/auth/login
```

Response

```
{
"token":"JWT_TOKEN"
}
```

Property APIs

Get All Properties

```
GET /api/properties
```

Get Property By ID

```
GET /api/properties/:id
```

Add Property

```
POST /api/properties
```

Booking APIs

Book Property

```
POST /api/bookings
```

View My Bookings

```
GET /api/bookings/my-bookings
```

---

8. Authentication

The application uses **JWT (JSON Web Token)** authentication.

Workflow:

1. User Registers
2. Password encrypted using bcrypt
3. User Logs In
4. JWT Token Generated
5. Token Stored in Local Storage
6. Protected Routes verify token before granting access

Security Features

- Password Hashing

- JWT Authentication
- Protected APIs
- Secure Route Access

---

9. User Interface

Include screenshots of:

- Home Page
- Login Page
- Registration Page
- Property Listing
- Property Details
- Add Property
- Dashboard
- My Bookings

Write 2–3 lines below each screenshot explaining its purpose.

---

10. Testing

Testing Strategy

The project was tested using manual testing techniques.

Module	Test Result
Registration	Passed
Login	Passed
Add Property	Passed
Search	Passed
Filter	Passed
Booking	Passed
Dashboard	Passed
Logout	Passed

---

11. Known Issues

- Payment Gateway is not integrated.
- Admin approval system is not implemented.
- Email notifications are unavailable.
- Property image upload is stored locally.

---

12. Future Enhancements

- Online Payment Gateway
- Admin Dashboard
- Google Maps Integration
- Property Reviews and Ratings
- Email Notifications
- SMS Notifications
- Cloudinary Image Upload
- AI-based Property Recommendation
- Live Chat Support
- Multi-language Support

Conclusion

The **House Rent Management System** successfully automates the house rental process using the MERN Stack. It provides a secure, scalable, and user-friendly platform for property owners and tenants. The application demonstrates full-stack web development concepts, including authentication, CRUD operations, REST APIs, database management, and responsive UI design.